Assignment 2. DOM-based To-Do List

- **Aim:** To build a dynamic, interactive To-Do List application using vanilla HTML, CSS, and JavaScript DOM manipulation.
- **Learning Objectives:** Understand Document Object Model (DOM) selection methods, event listeners (click, submit), and dynamic element creation/deletion.
- **Theoretical Background:** The DOM tree representing the page structure; using document.getElementById(), document.createElement(), appendChild(), and addEventListener() to modify UI state dynamically.
- **Detailed Requirements:**
    - **HTML Structure:** An input text field, an "Add Task" button, and an empty list container (<ul> or).
    - **CSS Styling:** Basic clean interface styling, including a visual indicator (like a strikethrough or color change) for completed tasks.
    - **JavaScript Logic:**
        - **Add:** Read the input field, create a new list item (<li>) dynamically, and append it to the container.
        - **Toggle Complete:** Click on a task to toggle a CSS class (e.g., .completed) that visually marks it off.
        - **Delete:** Add a "Delete" button next to each task that removes that specific node from the DOM when clicked.

Remark: _____

Start Date: _____ Date of Completion: _____

Signature of the Instructor: _____

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="theme-color" content="#f4f6fb">
  <title>To-Do List</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main class="todo-app">
    <header class="app-header">
      <p class="eyebrow">A little more focus</p>
      <h1>My to-do list</h1>
      <p class="subtitle">Write it down. Check it off. Keep going.</p>
    </header>

    <form class="task-form" id="task-form">
      <label class="visually-hidden" for="task-input">New task</label>
      <input
        id="task-input"
        name="task"
        type="text"
        placeholder="What needs to get done?"
        autocomplete="off"
        required
      >
      <button class="add-button" type="submit">Add task</button>
    </form>

    <section class="task-section" aria-labelledby="tasks-heading">
      <div class="list-heading">
        <h2 id="tasks-heading">Your tasks</h2>
        <span class="task-count" id="task-count" aria-live="polite">0 tasks</span>
      </div>
      <ul class="task-list" id="task-list" aria-label="To-do tasks"></ul>
      <p class="empty-state" id="empty-state">Nothing on your list yet. Add a task to get started.</p>
    </section>
  </main>

  <h2 style="margin-left: 40%;">Name : Akshay darji</h2>
  <h2 style="margin-left: 41%;">Prn : 1272250311</h2>

</body>
</html>
```

```js
const taskForm = document.getElementById("task-form");
const taskInput = document.getElementById("task-input");
const taskList = document.getElementById("task-list");
const taskCount = document.getElementById("task-count");
const emptyState = document.getElementById("empty-state");

function updateTaskSummary() {
  const count = taskList.childElementCount;
  taskCount.textContent = `${count} ${count === 1 ? "task" : "tasks"}`;
  emptyState.hidden = count > 0;
}

function createTask(text) {
  const item = document.createElement("li");
  item.className = "task-item";

  const toggleButton = document.createElement("button");
  toggleButton.className = "task-toggle";
  toggleButton.type = "button";
  toggleButton.setAttribute("aria-label", `Mark "${text}" as complete`);
  toggleButton.setAttribute("aria-pressed", "false");

  const taskText = document.createElement("span");
  taskText.className = "task-text";
  taskText.textContent = text;

  const deleteButton = document.createElement("button");
  deleteButton.className = "delete-button";
  deleteButton.type = "button";
  deleteButton.textContent = "Delete";
  deleteButton.setAttribute("aria-label", `Delete "${text}"`);

  toggleButton.addEventListener("click", () => {
    const isCompleted = item.classList.toggle("completed");
    toggleButton.setAttribute("aria-pressed", String(isCompleted));
    toggleButton.setAttribute(
      "aria-label",
      `Mark "${text}" as ${isCompleted ? "not complete" : "complete"}`
    );
  });

  taskText.addEventListener("click", () => toggleButton.click());

  deleteButton.addEventListener("click", () => {
    item.remove();
    updateTaskSummary();
  });

  item.append(toggleButton, taskText, deleteButton);
  return item;
}

taskForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const text = taskInput.value.trim();
  if (!text) {
    taskInput.focus();
    return;
  }

  taskList.appendChild(createTask(text));
  taskInput.value = "";
  updateTaskSummary();
  taskInput.focus();
});

updateTaskSummary();
```

```css
:root {
  color-scheme: light;
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #20263a;
  background: #f4f6fb;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

* {
  box-sizing: border-box;
}

body {
  min-width: 320px;
  min-height: 100vh;
  margin: 0;
  padding: 72px 20px;
}

button,
input {
  font: inherit;
}

.todo-app {
  width: min(100%, 560px);
  margin: 0 auto;
  padding: 42px;
  border: 1px solid #e7eaf2;
  border-radius: 20px;
  background: #fff;
  box-shadow: 0 18px 55px rgb(38 51 87 / 9%);
}

.app-header {
  margin-bottom: 30px;
}

.eyebrow {
  margin: 0 0 10px;
  color: #5c67c8;
  font-size: 0.75rem;
  font-weight: 750;
  letter-spacing: 0.13em;
  text-transform: uppercase;
}

h1,
h2,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 8px;
  font-size: clamp(2rem, 7vw, 2.5rem);
  letter-spacing: -0.055em;
}

.subtitle {
  margin-bottom: 0;
  color: #747b8f;
  font-size: 0.98rem;
}

.task-form {
  display: flex;
  gap: 10px;
  margin-bottom: 34px;
}

.task-item.completed .task-toggle {
  border-color: #5965cb;
  background: #5965cb;
}

.task-item.completed .task-toggle::after {
  content: "✓";
  font-size: 0.82rem;
  font-weight: 800;
}

.task-item.completed .task-text {
  color: #9699ab;
  text-decoration: line-through;
}

.delete-button {
  padding: 6px 8px;
  border: 0;
  border-radius: 7px;
  background: transparent;
  color: #8b91a1;
  cursor: pointer;
  font-size: 0.82rem;
}

.delete-button:hover {
  background: #fff0f0;
  color: #bd4545;
}

.empty-state {
  margin: 0;
  padding: 22px 12px;
  border: 1px dashed #dfe3ed;
  border-radius: 10px;
  color: #858b9b;
  font-size: 0.9rem;
  text-align: center;
}

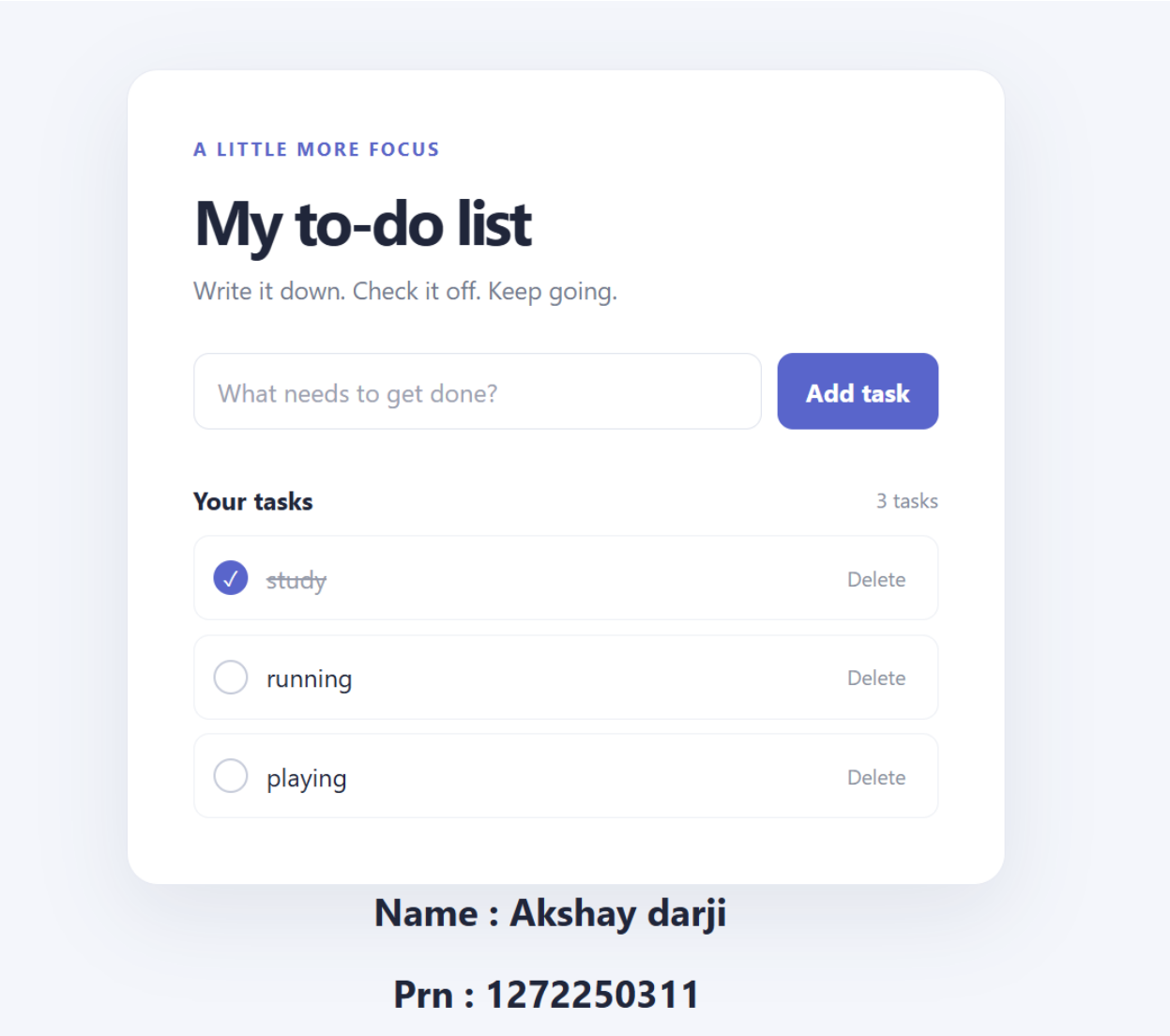
.empty-state[hidden] {
  display: none;
}

.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  clip-path: inset(50%);
}

@media (max-width: 480px) {
  body {
    padding: 28px 14px;
  }

  .todo-app {
    padding: 28px 20px;
  }

  .task-form {
    flex-direction: column;
  }
}
```



Name : Akshay darji

Prn : 1272250311

Assignment 7. REST API using Express.js

- **Aim:** To build a backend RESTful web service using Node.js and Express.js to handle standard HTTP requests.
- **Learning Objectives:** Initialize a Node.js project, handle npm packages, design Express routing, and manage JSON payloads.
- **Theoretical Background:** REST (Representational State Transfer) architecture uses HTTP verbs (GET, POST, PUT, DELETE) to perform operations on a resource.
- **Detailed Requirements:**
    - Set up an Express server listening on a local port (e.g., 5000).
    - Define an in-memory array of "User" objects (each containing id, name, and email) to act as a mock database.
    - Implement the following routes:
        - GET /api/users: Fetch all users.
        - POST /api/users: Add a new user (read data from req.body).
        - PUT /api/users/:id: Update user details matching the route parameter :id.
        - DELETE /api/users/:id: Remove the user matching the route parameter :id.
    - Test the endpoints using an API client (like Postman or Thunder Client).

Remark: _____

Start Date: _____ Date of Completion: _____

Signature of the Instructor: _____

{} package.json 4 ×

{} package.json > ...

```
{
  "name": "assignment7-express-api",
  "version": "1.0.0",
  "main": "server.js",
  "type": "module",
  ▷Debug
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.21.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  },
  "dependencies": {
    "express": "^5.3.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}

```

JS server.js > ...

```
import express from 'express';

const app = express();
const PORT = process.env.PORT || 5000;

app.use(express.json());

let users = [
  { id: 1, name: 'Kaustubh', email: 'kaustubh@example.com' },
  { id: 2, name: 'Chinmay', email: 'chinmay@example.com' }
];

app.get('/', (req, res) => {
  res.send('Welcome to the Assignment 7 Express REST API! Access endpoints at /api/users.');
});

app.get('/api/users', (req, res) => {
  res.status(200).json(users);
});

app.get('/api/users/:id', (req, res) => {
  const userId = parseInt(req.params.id, 10);
  const user = users.find((u) => u.id === userId);

  if (!user) {
    return res.status(404).json({ message: 'User not found' });
  }

  res.status(200).json(user);
});

app.post('/api/users', (req, res) => {
  const { name, email } = req.body;

  if (!name || !email) {
    return res.status(400).json({ message: 'Name and Email are required' });
  }

  const newUser = {
    id: users.length > 0 ? users[users.length - 1].id + 1 : 1,
    name,
    email
  };
```

```
  users.push(newUser);
  res.status(201).json({ message: 'User created successfully', user: newUser });
});

app.put('/api/users/:id', (req, res) => {
  const userId = parseInt(req.params.id, 10);
  const { name, email } = req.body;
  const userIndex = users.findIndex((u) => u.id === userId);

  if (userIndex === -1) {
    return res.status(404).json({ message: 'User not found' });
  }

  if (name) users[userIndex].name = name;
  if (email) users[userIndex].email = email;

  res.status(200).json({
    message: 'User updated successfully',
    user: users[userIndex]
  });
});

app.delete('/api/users/:id', (req, res) => {
  const userId = parseInt(req.params.id, 10);
  const userIndex = users.findIndex((u) => u.id === userId);

  if (userIndex === -1) {
    return res.status(404).json({ message: 'User not found' });
  }

  const deletedUser = users.splice(userIndex, 1)[0];
  res.status(200).json({
    message: 'User deleted successfully',
    user: deletedUser
  });
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Pretty print ☑


```
[
  {
    "_id": "6adca31fd093889e65ccbbdf",
    "name": "akki",
    "email": "akki123@gmail.com",
    "createdAt": "2026-10-24T12:22:55.913Z",
    "updatedAt": "2026-10-24T12:22:55.913Z",
    "__v": 0
  },
  {
    "_id": "6adb13b6bf1c51aaed380061",
    "name": "Aarav",
    "email": "aarav@example.com",
    "createdAt": "2026-10-23T07:58:46.654Z",
    "updatedAt": "2026-10-23T07:58:46.654Z",
    "__v": 0
  },
  {
    "_id": "6adb1238bf1c51aaed38005e",
    "name": "Alex Smith",
    "email": "alex@gmail.com",
    "createdAt": "2026-10-23T07:52:24.451Z",
    "updatedAt": "2026-10-23T07:52:24.451Z",
    "__v": 0
  }
]
```

Assignment 8. MongoDB Integration using Mongoose

- **Aim:** To establish a database connection between an Express application and MongoDB using Mongoose, executing basic CRUD operations.
- **Learning Objectives:** Understand NoSQL database concepts, design structured schemas using Mongoose, and handle asynchronous database operations (async/await).
- **Theoretical Background:** Mongoose provides a straight-forward, schema-based solution to model application data with MongoDB.
- **Detailed Requirements:**
    - Configure a local MongoDB instance or a MongoDB Atlas cloud database.
    - Define a strict Mongoose Schema for a Product (fields: name [String], price [Number], category [String], stock [Number]).
    - Create Express routes to:
        - Create a product (Save to MongoDB).
        - Read products (Find all / Find by ID).
        - Update a product's price or stock.
        - Delete a product by ID.
    - Implement error-handling middleware for database connection failures or missing parameters.

Remark: _____

Start Date: _____ Date of Completion: _____

Signature of the Instructor: _____

package.json > ...

```json
{
  "name": "assignment8-mongoose-api",
  "version": "1.0.0",
  "description": "Assignment 8: MongoDB & Mongoose Integration",
  "main": "server.js",
  "type": "module",
  ▷Debug
  "scripts": {
    "start": "node --env-file=.env server.js",
    "dev": "nodemon --env-file=.env server.js"
  },
  "dependencies": {
    "dotenv": "^18.0.9",
    "express": "^5.2.1",
    "mongoose": "^9.11.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

JS server.js > ...

```
import 'dotenv/config';
import express from 'express';
import mongoose from 'mongoose';

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware to parse incoming JSON request bodies
app.use(express.json());

// ==========================================
// 1. DATABASE CONNECTION
// ==========================================
const PRIMARY_URI = process.env.MONGO_URI || 'mongodb+srv://kaustubhkeer_db_user:mcapassword123@cluster0.x9iddmu.mongodb.net/mca_product_db?retryWrites=true&w=majority';
const LOCAL_FALLBACK_URI = 'mongodb://127.0.0.1:27017/mca_product_db';

const connectDB = async () => {
  try {
    await mongoose.connect(PRIMARY_URI, { serverSelectionTimeoutMS: 4000 });
    console.log(`✅ Connected successfully to MongoDB (${PRIMARY_URI.includes('mongodb+srv') ? 'Atlas Cloud' : 'Local'})`);
  } catch (err) {
    console.warn(`⚠️ Primary MongoDB connection failed (${err.name}): ${err.message}`);
    if (PRIMARY_URI !== LOCAL_FALLBACK_URI) {
      console.log('🔄 Attempting fallback to local MongoDB (mongodb://127.0.0.1:27017/mca_product_db)...');
      try {
        await mongoose.connect(LOCAL_FALLBACK_URI, { serverSelectionTimeoutMS: 4000 });
        console.log('✅ Connected successfully to Local MongoDB');
      } catch (fallbackErr) {
        console.error('❌ Local MongoDB Connection Error:', fallbackErr.message);
      }
    }
  }
};

connectDB();

// ==========================================
// 2. MONGOOSE SCHEMA & MODEL DEFINITION
// ==========================================
const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Product name is required'],
    trim: true
  },
  price: {
    type: Number,
    required: [true, 'Product price is required'],
    min: [0, 'Price cannot be negative']
  },
  category: {
    type: String,
    required: [true, 'Category is required']
  },
  stock: {
    type: Number,
    default: 0,
    min: [0, 'Stock cannot be negative']
  }
}, { timestamps: true });

const Product = mongoose.model('Product', productSchema);

// ==========================================
// 3. REST API ROUTES (CRUD)
```

```
// Root Route
app.get('/', (req, res) => {
  res.send('Welcome to Assignment 8: Mongoose Product API!');
});

// CREATE: Add a new product (POST /api/products)
app.post('/api/products', async (req, res, next) => {
  try {
    const { name, price, category, stock } = req.body;
    const newProduct = new Product({ name, price, category, stock });
    const savedProduct = await newProduct.save();
    res.status(201).json({
      message: 'Product created successfully',
      product: savedProduct
    });
  } catch (error) {
    next(error);
  }
});

// READ ALL: Fetch all products (GET /api/products)
app.get('/api/products', async (req, res, next) => {
  try {
    const products = await Product.find();
    res.status(200).json(products);
  } catch (error) {
    next(error);
  }
});

// READ ONE: Fetch product by ID (GET /api/products/:id)
app.get('/api/products/:id', async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) {
      return res.status(404).json({ message: 'Product not found' });
    }
    res.status(200).json(product);
  } catch (error) {
    next(error);
  }
});

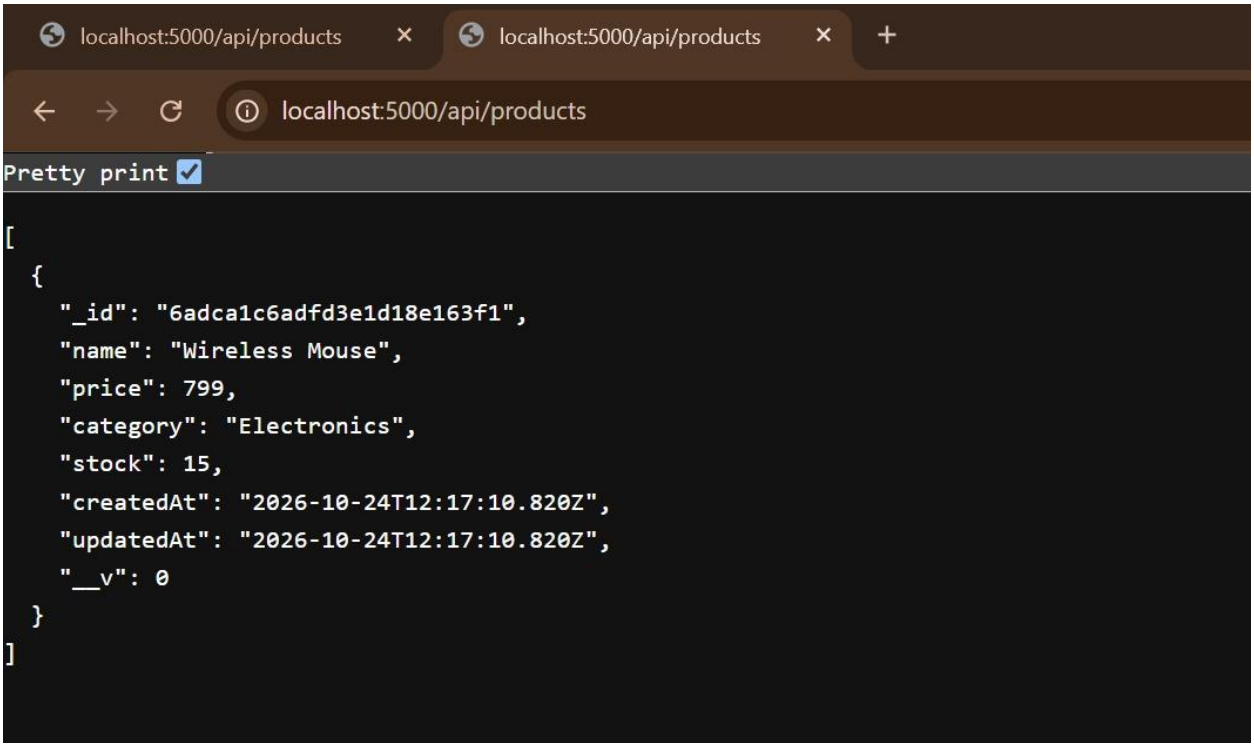
// UPDATE: Modify product details (PUT /api/products/:id)
app.put('/api/products/:id', async (req, res, next) => {
  try {
    const updatedProduct = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!updatedProduct) {
      return res.status(404).json({ message: 'Product not found' });
    }
    res.status(200).json({
      message: 'Product updated successfully',
      product: updatedProduct
    });
  } catch (error) {
    next(error);
  }
```

```
    next(error);
  }
});

// DELETE: Remove product by ID (DELETE /api/products/:id)
app.delete('/api/products/:id', async (req, res, next) => {
  try {
    const deletedProduct = await Product.findByIdAndDelete(req.params.id);
    if (!deletedProduct) {
      return res.status(404).json({ message: 'Product not found' });
    }
    res.status(200).json({
      message: 'Product deleted successfully',
      product: deletedProduct
    });
  } catch (error) {
    next(error);
  }
});

// ========================================
// 4. CENTRALIZED ERROR HANDLING MIDDLEWARE
// ========================================
app.use((err, req, res, next) => {
  console.error('Error Stack:', err.message);
  if (err.name === 'CastError') {
    return res.status(400).json({ message: 'Invalid ID format provided' });
  }
  if (err.name === 'ValidationError') {
    return res.status(400).json({ message: err.message });
  }
  res.status(500).json({ message: 'Internal Server Error', error: err.message });
});

// Start Server
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
});
```





Welcome to Assignment 8: Mongoose Product API!

Assignment 9. Full-Stack Integration (React + Express + MongoDB)

- **Aim:** To connect the React frontend with the Node/Express backend and MongoDB database to form a complete MERN application.
- **Learning Objectives:** Master Cross-Origin Resource Sharing (CORS) configurations, execute frontend HTTP requests using Axios, and integrate full-stack components.
- **Theoretical Background:** A full-stack application connects a client layer (React) to a controller/server layer (Express) which persists data to a storage layer (MongoDB).
- **Detailed Requirements:**
    - **Backend:**
        - Create an Express server connected to a MongoDB database containing a "User" collection.
        - Expose endpoints to get users and post a new user.
        - Enable cors middleware to allow requests from your React frontend port.
    - **Frontend:**
        - A React form to input user details (name, email).
        - Use **Axios** (or the Fetch API) inside a useEffect hook to pull the existing list of users from the backend upon mounting.
        - Submit the form to trigger an Axios POST request, saving the user to the database, and dynamically updating the UI list with the newly added user.

Remark: _____

Start Date: _____ Date of Completion: _____

Signature of the Instructor: _____

server > {} package.json > ...

```
{
  "name": "assignment9-server",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongoose": "^8.7.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}

```

server > JS server.js > ...

```
import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';
import dotenv from 'dotenv';

dotenv.config();

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/assignment9';

app.use(cors());
app.use(express.json());

mongoose.set('strictQuery', true);

mongoose
  .connect(MONGO_URI)
  .then(() => console.log('✅ Connected successfully to MongoDB'))
  .catch((err) => console.error('❌ MongoDB Connection Error:', err.message));

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      trim: true,
      lowercase: true,
    },
  },
  { timestamps: true }
);

const User = mongoose.model('User', userSchema);

app.get('/', (req, res) => {
  res.send('Assignment 9: Full-Stack Integration Server is running.');
});

app.get('/api/users', async (req, res) => {
  try {
    const users = await User.find().sort({ createdAt: -1 });
    res.status(200).json(users);
  } catch (error) {
    res.status(500).json({ message: 'Error retrieving users', error: error.message });
  }
});
```

```
app.post('/api/users', async (req, res) => {
  try {
    const { name, email } = req.body;

    if (!name || !email) {
      return res.status(400).json({ message: 'Both Name and Email are required fields.' });
    }

    const newUser = new User({
      name: name.trim(),
      email: email.trim().toLowerCase(),
    });

    const savedUser = await newUser.save();
    res.status(201).json(savedUser);
  } catch (error) {
    if (error.code === 11000) {
      return res.status(400).json({ message: 'An account with this email already exists.' });
    }

    res.status(400).json({ message: error.message });
  }
});

app.listen(PORT, () => {
  console.log(`🚀 Express server running on http://localhost:${PORT}`);
});

```

client > src > App.jsx > ...

```jsx
import { useState, useEffect } from 'react';
import axios from 'axios';

const API_URL = 'http://localhost:5000/api/users';

function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const fetchUsers = async () => {
    try {
      setLoading(true);
      setError('');
      const response = await axios.get(API_URL);
      setUsers(response.data);
    } catch (err) {
      console.error('Error fetching users:', err);
      setError('Unable to load users. Ensure backend server is running on port 5000.');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchUsers();
  }, []);

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!name.trim() || !email.trim()) return;

    try {
      setError('');
      const response = await axios.post(API_URL, {
        name: name.trim(),
        email: email.trim(),
      });

      setUsers((currentUsers) => [response.data, ...currentUsers]);
      setName('');
      setEmail('');
    } catch (err) {
      console.error('Error saving user:', err);
      const serverMsg = err.response?.data?.message || 'Failed to submit user data.';
      setError(serverMsg);
    }
  };

  return (
    <div style={styles.container}>
      <header style={styles.header}>
        <h2>Assignment 9: Full-Stack User Directory</h2>
        <p style={styles.subtext}>React (Port 5173) + Express (Port 5000) + MongoDB Atlas</p>
      </header>

      <form onSubmit={handleSubmit} style={styles.form}>
        <h3>Add New User</h3>
        <input
          type="text"
          placeholder="Full Name"
          value={name}
```

```
          onChange={(e) => setName(e.target.value)}
          required
          style={styles.input}
        />
        <input
          type="email"
          placeholder="Email Address"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
          style={styles.input}
        />
        <button type="submit" style={styles.button}>
          Submit to Database
        </button>
      </form>

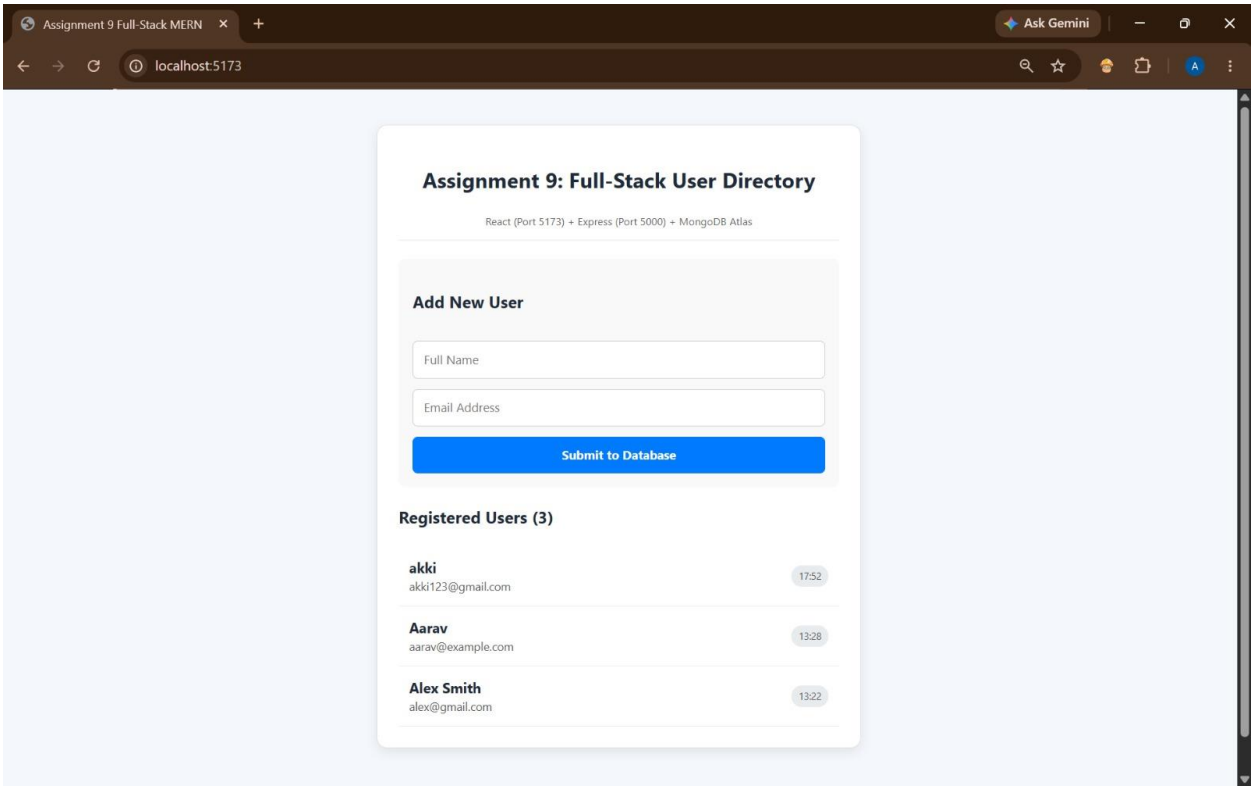
      {error && <div style={styles.errorBanner}>{error}</div>}

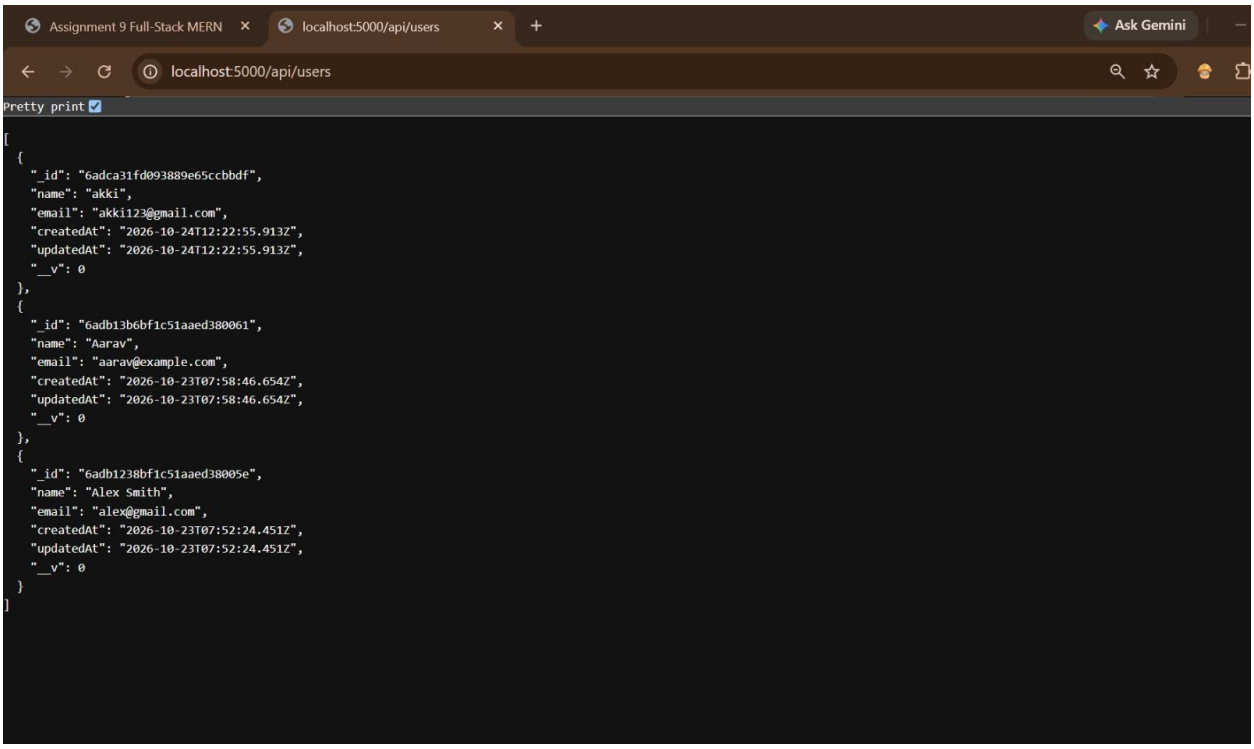
      <section style={styles.listSection}>
        <h3>Registered Users ({users.length})</h3>
        {loading ? (
          <p style={styles.infoText}>Loading records from MongoDB Atlas...</p>
        ) : users.length === 0 ? (
          <p style={styles.infoText}>No user records found in database.</p>
        ) : (
          <ul style={styles.list}>
            {users.map((user) => (
              <li key={user._id} style={styles.listItem}>
                <div>
                  <strong>{user.name}</strong>
                  <div style={styles.emailText}>{user.email}</div>
                </div>
                <span style={styles.badge}>
                  {new Date(user.createdAt).toLocaleTimeString([], {
                    hour: '2-digit',
                    minute: '2-digit',
                  })}
                </span>
              </li>
            ))}
          </ul>
        )}
      </section>
    </div>
  );
}

const styles = {
  container: {
    maxWidth: '550px',
    margin: '40px auto',
    fontFamily: 'Segoe UI, Tahoma, Geneva, Verdana, sans-serif',
    padding: '24px',
    border: '1px solid #e0e0e0',
    borderRadius: '12px',
    boxShadow: '0 4px 12px rgba(0,0,0,0.08)',
    backgroundColor: '#ffffff',
  },
  header: {
    textAlign: 'center',
    borderBottom: '2px solid #f0f0f0',
    paddingBottom: '12px',
    marginBottom: '20px',
```

```
  },
  input: {
    padding: '10px 12px',
    fontSize: '14px',
    borderRadius: '6px',
    border: '1px solid #ccc',
    outline: 'none',
  },
  button: {
    padding: '10px',
    fontSize: '14px',
    fontWeight: 'bold',
    color: '#fff',
    backgroundColor: '#007bff',
    border: 'none',
    borderRadius: '6px',
    cursor: 'pointer',
  },
  errorBanner: {
    padding: '10px',
    backgroundColor: '#ffe6e6',
    color: '#d9534f',
    border: '1px solid #f5c6cb',
    borderRadius: '6px',
    marginBottom: '16px',
    fontSize: '13px',
  },
  listSection: {
    marginTop: '10px',
  },
  list: {
    listStyleType: 'none',
    padding: 0,
    margin: 0,
  },
  listItem: {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '12px',
    borderBottom: '1px solid #eee',
  },
  emailText: {
    fontSize: '13px',
    color: '#555',
  },
  badge: {
    fontSize: '11px',
    backgroundColor: '#e9ecef',
    padding: '4px 8px',
    borderRadius: '12px',
    color: '#495057',
  },
  infoText: {
    color: '#777',
    fontStyle: 'italic',
    textAlign: 'center',
  },
};

export default App;
```






```
[
  {
    "_id": "6adca31fd093889e65ccbbdf",
    "name": "akki",
    "email": "akki123@gmail.com",
    "createdAt": "2026-10-24T12:22:55.913Z",
    "updatedAt": "2026-10-24T12:22:55.913Z",
    "__v": 0
  },
  {
    "_id": "6adb13b6bf1c51aaed380061",
    "name": "Aarav",
    "email": "aarav@example.com",
    "createdAt": "2026-10-23T07:58:46.654Z",
    "updatedAt": "2026-10-23T07:58:46.654Z",
    "__v": 0
  },
  {
    "_id": "6adb1238bf1c51aaed38005e",
    "name": "Alex Smith",
    "email": "alex@gmail.com",
    "createdAt": "2026-10-23T07:52:24.451Z",
    "updatedAt": "2026-10-23T07:52:24.451Z",
    "__v": 0
  }
]
```